

Consola Retro SDK — Sistema, Instalación Automatizada e Integración

Fundamentos de Sistemas Embebidos

Tutorial individual — Módulos: *usb_monitor.py* · *main.py* · *instalar.sh*

1. Objetivo

Implementar la integración principal de una consola retro embebida en Raspberry Pi, asegurando su arranque automático, detección de memorias USB y ejecución funcional de los módulos del sistema.

Desarrollar e integrar los módulos *main.py*, *usb_monitor.py* e *instalar.sh* para coordinar el inicio del sistema, automatizar la instalación de dependencias, configurar el servicio de arranque con systemd y permitir la detección y copia de ROMs desde una memoria USB de forma segura.

2. Lista de materiales

Componente	Notas
Raspberry Pi 4 Model B (2 GB o más)	Mínimo 2 GB RAM recomendados
Tarjeta microSD (16 GB+, clase 10)	Clase 10 o UHS-I para velocidad
Monitor o TV con entrada HDMI	Resolución mínima 720p
Cable micro-HDMI a HDMI	Incluido en algunos kits
Fuente de alimentación 5 V / 3 A USB-C	Oficial Raspberry Pi recomendada
Memoria USB (FAT32 o exFAT)	Para carga de ROMs adicionales
Computadora con acceso a internet	Para preparar la tarjeta SD
Gamepad USB o Bluetooth compatible	Compatible con SDL2 (ej. Xbox, genérico)

Tabla 1. Hardware requerido

En cuanto al software se requiere:

- Raspberry Pi OS Lite (64-bit), sin entorno de escritorio
- Python 3.11 o superior
- pygame 2.x (instalado vía apt: python3-pygame)
- pyudev — bindings Python para libudev (python3-pyudev)
- mednafen — emulador multi-sistema (mednafen)
- git — para clonar el repositorio durante la instalación
- curl y unzip — para descargar y extraer assets durante la instalación
- systemd — gestor de servicios (incluido en Raspberry Pi OS)

3. Antecedentes teóricos

3.1 Integración de sistemas embebidos

Un sistema embebido es un sistema de cómputo diseñado para cumplir una función específica dentro de un dispositivo mayor. A diferencia de una computadora de propósito general, normalmente trabaja con recursos limitados y debe operar de manera autónoma, estable y confiable. [1].

Un principio fundamental de diseño en sistemas embebidos es que el software debe arrancar de forma autónoma sin intervención humana al recibir alimentación. Esto requiere configurar el sistema operativo para que lance la aplicación automáticamente como parte del proceso de arranque, antes de que cualquier sesión de usuario esté disponible [1].

3.2 systemd y servicios de arranque

Para lograr que la consola se ejecute automáticamente al encender la Raspberry Pi, se utiliza systemd, el sistema de inicialización y administración de servicios usado en muchas distribuciones Linux modernas. Mediante un archivo de servicio, es posible indicar qué programa debe ejecutarse, con qué usuario, desde qué directorio y qué hacer si el proceso falla. Esto es importante en el proyecto porque permite que la consola arranque sin intervención manual y que pueda reiniciarse automáticamente si ocurre un error [2].

3.3 udev y detección de hardware en Linux

Otro elemento importante es la detección de memorias USB. Linux administra los dispositivos conectados mediante udev, el cual genera eventos cuando se conecta o desconecta hardware. La biblioteca pyudev permite recibir esos eventos desde Python, por lo que el programa puede detectar una memoria USB en tiempo real sin estar revisando constantemente los archivos del sistema.

Este enfoque basado en eventos es más eficiente que un sondeo continuo, ya que reduce el uso innecesario de CPU [3].

La ventaja de este enfoque sobre el polling (revisar periódicamente si aparecen dispositivos nuevos) es que consume recursos mínimos mientras no hay eventos: el hilo de monitoreo queda bloqueado en `monitor.poll()` hasta que el kernel genera un evento, sin ciclos de CPU desperdiciados.

3.4 Montaje de sistemas de archivos en Linux

Cuando se detecta una memoria USB, el sistema debe montarla para acceder a su contenido. En Linux, montar un dispositivo significa asociarlo con una carpeta del sistema de archivos para poder leer sus datos. En este proyecto, la memoria se monta en modo de solo lectura para copiar ROMs hacia la Raspberry Pi sin modificar el contenido original del USB. Posteriormente, el dispositivo se desmonta para evitar errores o corrupción de archivos [4].

3.5 Programación con hilos y exclusión mutua

El monitoreo de memorias USB debe ejecutarse sin detener la interfaz principal de la consola. Para lograrlo, el programa utiliza hilos. Un hilo permite ejecutar una tarea en segundo plano mientras el resto del programa continúa funcionando. En este caso, el hilo de monitoreo espera eventos USB, mientras el hilo principal mantiene activa la galería y la interacción con el usuario [5].

El uso de hilos implica cuidar el acceso a variables compartidas. Si dos hilos leen o modifican una misma variable al mismo tiempo, pueden ocurrir errores difíciles de detectar, conocidos como condiciones de carrera. En el proyecto, la variable que indica si se conectó un nuevo USB debe compartirse entre el hilo de monitoreo y el ciclo principal de la galería.

Para evitar este problema se utiliza `threading.Lock`. Este mecanismo funciona como un seguro: antes de modificar o leer una variable compartida, el hilo debe adquirir el bloqueo. Cuando termina, lo libera. De esta manera se garantiza que solo un hilo acceda a esa información a la vez, manteniendo la comunicación entre módulos de forma segura.

3.6 Scripts bash de instalación automatizada

Un script bash de instalación automatizada permite ejecutar de forma ordenada los comandos necesarios para preparar un sistema. En lugar de instalar dependencias, crear carpetas, copiar archivos y configurar servicios manualmente, el script reúne todos esos pasos en un solo archivo ejecutable [4].

En este proyecto, `instalar.sh` se encarga de configurar la Raspberry Pi desde una instalación limpia de Raspberry Pi OS Lite. El script instala paquetes necesarios como Python, pygame, pyudev y mednafen; crea la estructura de directorios del proyecto; configura permisos; prepara los archivos de Mednafen; y registra el servicio de systemd para que la consola arranque automáticamente.

Este tipo de automatización es útil porque reduce errores humanos y permite repetir la instalación de forma consistente en diferentes tarjetas microSD o Raspberry Pi. Además, facilita que cualquier integrante del equipo pueda obtener el mismo entorno de ejecución siguiendo un proceso único y controlado. Dentro del script se utiliza la directiva `set -euo pipefail`, la cual ayuda a que la instalación sea más segura. La opción `-e` detiene el script si un comando falla, `-u` evita el uso de variables no definidas y `pipefail` permite detectar errores dentro de comandos encadenados. Estas opciones ayudan a que los problemas se detecten temprano y no se acumulen durante la instalación.

4. Advertencias y cuidado de componentes

4.1 Seguridad durante la instalación

El script `instalar.sh` debe ejecutarse con permisos de administrador, ya que necesita instalar dependencias, crear directorios del sistema, modificar permisos y registrar el servicio de arranque automático. Por esta razón, se recomienda revisar el contenido del script antes de ejecutarlo con `sudo`, especialmente porque realiza cambios en rutas importantes del sistema operativo.

Durante la instalación se modifican archivos relacionados con permisos, servicios de systemd y parámetros de arranque. Una configuración incorrecta podría provocar que la consola no inicie correctamente o que el sistema requiera mantenimiento manual. Para evitar pérdida de información, es recomendable realizar una copia de seguridad de la tarjeta microSD antes de ejecutar el proceso de instalación.

También se recomienda habilitar previamente el acceso SSH o tener conectados un teclado y un monitor a la Raspberry Pi. Esto permite recuperar el control del sistema en caso de que el servicio de la consola falle, no arranque correctamente o sea necesario revisar archivos de configuración.

4.2 Seguridad de la tarjeta microSD

El sistema está configurado para apagarse siempre mediante `sudo shutdown -h now` antes de cortar la alimentación. Este paso es crítico para preservar la integridad del sistema de archivos en la microSD.

- El servicio de arranque incluye una condición que permite deshabilitar temporalmente la consola mediante un archivo de control llamado `NO_CONSOLA`. Esto es útil cuando se necesita acceder al sistema para mantenimiento, actualizar archivos o corregir errores sin que la aplicación principal se ejecute automáticamente al iniciar.

4.3 Cuidado con memorias USB

Las memorias USB se utilizan para cargar nuevas ROMs al sistema. Para mejorar la compatibilidad, se recomienda formatearlas en FAT32 o exFAT, ya que son sistemas de archivos comúnmente reconocidos por Raspberry Pi OS y por otros sistemas operativos.

El sistema monta la memoria USB en modo de solo lectura. Esto significa que la Raspberry Pi puede leer y copiar archivos desde el USB, pero no modifica el contenido original de la memoria. Esta decisión reduce el riesgo de borrar archivos accidentalmente o dañar la información almacenada en el dispositivo.

Después de detectar y copiar las ROMs, el sistema desmonta la memoria USB para finalizar correctamente el proceso. Se recomienda retirar la memoria únicamente después de que la consola haya terminado la copia y actualizado la galería de juegos. Retirar el USB antes de tiempo puede provocar que algunos archivos no se copien correctamente.

Además, se recomienda organizar las ROMs con nombres claros y evitar archivos duplicados. Aunque el sistema compara los nombres de los archivos para no copiar ROMs repetidas, mantener una estructura ordenada en la memoria USB facilita la revisión y reduce errores durante las pruebas.

5. Descripción de los módulos de software

El proyecto se organiza en diferentes carpetas y archivos para separar la configuración, los módulos principales, los recursos gráficos, los sonidos, las ROMs y el script de instalación. Esta estructura permite que cada parte del sistema tenga una función específica y facilita su mantenimiento.

5.1 Estructura del proyecto

Ruta	Contenido
src/main.py	Punto de entrada del sistema; instancia y conecta todos los módulos principales.
src/config/configuracion.json	Archivo de configuración general: resolución, rutas de assets y mapeo de emuladores.
src/config/mednafen/mednafen.cfg	Configuración pre-ajustada de mednafen para Raspberry Pi
src/roms/nes/	Carpeta donde se almacenan las ROMs de NES con extensión .nes.
src/roms/snes/	Carpeta donde se almacenan las ROMs de Super Nintendo con extensión .smc o .sfc.
src/roms/gba/	Carpeta donde se almacenan las ROMs de Game Boy Advance con extensión .gba.
src/assets/imagenes/	Logo de arranque, imágenes de controles por consola

src/assets/sonidos/	Archivos de audio utilizados durante el arranque o la navegación.
src/instalar.sh	Script de instalación automatizada completa

Tabla 2. Estructura de directorios del proyecto

5.2 Módulo main.py — punto de entrada

main.py es el único archivo que se ejecuta directamente. Su responsabilidad es instanciar todos los módulos en el orden correcto y conectar sus dependencias. No contiene lógica de negocio: delega toda la funcionalidad a los módulos especializados.

La secuencia de inicialización en main() es la siguiente:

1. `instalar_config_mednafen()`: copia el archivo de configuración de mednafen al directorio `~/mednafen` en cada arranque, garantizando que siempre se use la configuración correcta.
2. Llana a `cargar_configuracion()`: lee el archivo JSON de configuración y valida su existencia y formato.
3. Crea la instancia de Pantalla usando los parámetros de resolución definidos en el JSON.
4. Crea las instancias de Entrada y MonitorUSB.
5. Crea la instancia de Emulador, pasándole la pantalla, la ruta de imágenes y el mapeo de emuladores
6. Registrar la referencia del Emulador en MonitorUSB con `set_emulador()`.
7. Instanciar Galeria con referencias a todos los módulos anteriores.
8. Instanciar Arranque con rutas al logo y sonido del JSON y ejecutar la secuencia de arranque.
9. Iniciar el loop principal con `galeria.ejecutar()`, que no retorna hasta el apagado.
10. Al finalizar, ejecuta `pantalla.cerrar()` para liberar correctamente los recursos gráficos.

5.3 Módulo usb_monitor.py — Clase MonitorUSB

La clase MonitorUSB es el componente más complejo del subsistema de integración. Gestiona la detección de memorias USB, su montaje, la copia de ROMs y la comunicación con el loop principal y el emulador, todo en un hilo separado para no interferir con la interfaz gráfica.

Método	Descripción
<code>__init__()</code>	Crea directorio de montaje, inicia hilo de monitoreo
<code>set_emulador(emulador, ruta)</code>	Registra referencia al emulador para detenerlo si hay USB durante el juego
<code>hay_nuevo_usb()</code>	Devuelve True y resetea bandera si hay USB listo para procesar
<code>copiar_roms_de_usb(ruta_local)</code>	Copia ROMs del USB al proyecto evitando duplicados; desmonta al terminar
<code>_monitorear_udev()</code>	Hilo principal: escucha eventos de bloque con pyudev

<code>_monitorear_polling()</code>	Método alternativo que revisa periódicamente dispositivos <code>/dev/sd*</code> si <code>pyudev</code> no está disponible.
<code>_montar(dispositivo)</code>	Monta el dispositivo USB en punto de montaje fijo
<code>_desmontar()</code>	Desmonta el USB de forma segura después de copiar
<code>_buscar_archivos(ruta, exts)</code>	Búsqueda recursiva de archivos por extensión con glob

Tabla 3. Métodos de la clase MonitorUSB

5.3.1 Arquitectura de doble hilo

MonitorUSB trabaja con dos hilos principales. El primero es el hilo principal de la aplicación, donde se ejecuta la interfaz de la galería. El segundo es un hilo de monitoreo que permanece atento a la conexión de memorias USB.

Esta arquitectura evita que la consola se congele mientras espera la inserción de un dispositivo. El hilo de monitoreo detecta el USB y actualiza una bandera interna llamada `_usb_nuevo`, mientras que la galería revisa esa bandera desde su ciclo principal mediante el método `hay_nuevo_usb()`.

5.3.2 Fallback de polling

Este método revisa periódicamente la existencia de dispositivos como `/dev/sda1` o `/dev/sdb1`. Aunque consume más recursos que el enfoque basado en eventos, permite que el sistema siga funcionando en entornos donde `pyudev` no pueda utilizarse correctamente.

5.4 Script `instalar.sh` — instalación automatizada

El script `instalar.sh` automatiza la configuración del sistema en una instalación limpia de Raspberry Pi OS Lite. Su objetivo es evitar que el usuario tenga que realizar manualmente todos los pasos de instalación, configuración de dependencias, creación de carpetas, permisos y registro del servicio de arranque.

Paso	Acción	Descripción
0	Actualizar sistema	<code>apt update</code> e instalación de <code>git</code> , <code>python3</code> , <code>curl</code>
1	Dependencias SDL	<code>libsdl2</code> , <code>libsdl2-mixer</code> , <code>libsdl2-image</code> , <code>libsdl2-ttf</code>
2	<code>pygame</code> y <code>pyudev</code>	<code>python3-pygame</code> , <code>python3-pyudev</code> vía <code>apt</code>
3	<code>mednafen</code>	Instalación y enlace simbólico en <code>/usr/local/bin</code>
4	Permisos de usuario	Agregar usuario a grupos <code>video</code> , <code>audio</code> , <code>input</code> , <code>render</code>
5	Estructura de directorios	Crear carpetas <code>roms/nes</code> , <code>roms/snes</code> , <code>roms/gba</code> , <code>.mednafen</code>
6	Código fuente	Clonar repositorio GitHub y copiar archivos <code>.py</code>
7	Assets y ROMs	Descargar y descomprimir desde branch instalaciones
8	Sudoers	Permitir <code>mount</code> , <code>umount</code> y <code>shutdown</code> sin contraseña
9	Configuración <code>mednafen</code>	Copiar <code>mednafen.cfg</code> preconfigurado a <code>~/.mednafen</code>
10	Variables SDL en <code>.bashrc</code>	<code>SDL_VIDEODRIVER=kmsdrm</code> , <code>SDL_AUDIODRIVER=alsa</code>

11	Servicio systemd	Crear y habilitar consola-retro.service con autoarranque
12	TTY de mantenimiento	Habilitar getty@tty2 para acceso con Ctrl+Alt+F2
13	Silenciar arranque	Modifica parámetros de arranque para ocultar mensajes del sistema. (Aún con fallos)

Tabla 4. Pasos del script de instalación instalar.sh

6. Configuración de la Raspberry Pi

6.1 Preparación de la tarjeta microSD

Para iniciar la instalación del sistema se utiliza una tarjeta microSD con Raspberry Pi OS Lite de 64 bits:

- Nombre de host: consola-retro (o el deseado)
- Nombre de usuario y contraseña
- Red Wi-Fi (SSID y contraseña) para acceso SSH remoto
- Habilitar SSH con autenticación por contraseña

Una vez configurada la tarjeta, se inserta en la Raspberry Pi, se conecta la alimentación y se espera a que arranque. Se puede acceder por SSH con:

```
ssh usuario@consola-retro.local
```

6.2 Ejecución del script de instalación

Con acceso SSH, se ejecuta el script de instalación directamente desde el repositorio:

```
sudo bash -c "$(curl -fsSL https://raw.githubusercontent.com/nasarukey/sdk_retrogames/main/src/instalar.sh)"
```

El uso de sudo es necesario porque el script realiza cambios en rutas del sistema, instala paquetes mediante apt, configura permisos especiales y crea un servicio dentro de systemd.

Durante la ejecución, el script identifica el usuario real del sistema, incluso si se ejecuta con privilegios de administrador. Esto permite que los archivos del proyecto queden asignados al usuario correcto y no solamente al usuario root.

6.3 Archivo de configuración JSON

El sistema se configura mediante el archivo src/config/configuracion.json. Los parámetros más relevantes son:

```
{
  "pantalla": {
    "ancho": 1280,
    "alto": 720,
    "pantalla_completa": true
  },
  "emuladores": {
    "nes": "mednafen",
    "snes": "mednafen",
    "gba": "mednafen"
  }
}
```

```

    },
    "rutas": {
        "roms": "roms",
        "imagenes": "assets/imagenes",
        "sonidos": "assets/sonidos"
    }
}

```

6.4 Servicio systemd

El archivo `/etc/systemd/system/consola-retro.service` define el autoarranque. Los aspectos más importantes de su configuración son:

```

[Unit]
Description=Consola Retro SDK
After=multi-user.target sound.target local-fs.target
Wants=sound.target
ConditionPathExists=!/boot/NO_CONSOLA
ConditionPathExists=!/boot/firmware/NO_CONSOLA

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/consola_retro/src
Environment=HOME=/home/pi
Environment=SDL_VIDEODRIVER=kmsdrm
Environment=SDL_AUDIODRIVER=alsa
Environment=PYTHONUNBUFFERED=1
ExecStart=/usr/bin/python3 /home/pi/consola_retro/src/main.py
StandardOutput=append:/home/pi/consola_retro.log
StandardError=append:/home/pi/consola_retro.log
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target

```

La directiva `After=local-fs.target` garantiza que los sistemas de archivos estén montados antes de que la consola intente acceder a sus archivos. `Restart=always` con `RestartSec=3` reinicia la consola 3 segundos después de cualquier fallo, sin intervención humana.

6.5 Silenciado del arranque de Linux

Para que la consola presente una experiencia limpia, se suprimen los mensajes del kernel y del sistema de arranque agregando parámetros al archivo `/boot/firmware/cmdline.txt`:

```

quiet loglevel=0 logo.nologo fsck.mode=skip
rd.systemd.show_status=false rd.udev.log_level=3

```

Adicionalmente se vacían los archivos `/etc/motd`, `/etc/issue` y `/etc/issue.net` para eliminar los mensajes de bienvenida del sistema.

6.6 TTY de mantenimiento

Aunque el objetivo es que la Raspberry Pi funcione como una consola dedicada, también es importante conservar una forma de acceder al sistema para mantenimiento. Para esto se habilita una terminal secundaria mediante `getty@tty2`.

Esta terminal puede abrirse con la combinación: *Ctrl + Alt + F2*.

Desde ahí es posible iniciar sesión, revisar archivos, detener el servicio, consultar logs o realizar cambios de configuración sin depender únicamente de SSH.

7. Desarrollo de los módulos

El archivo `main.py` funciona como el punto de entrada del sistema. Antes de iniciar la interfaz de la consola, este módulo se encarga de preparar la configuración general y crear las instancias necesarias para que los demás módulos trabajen en conjunto.

7.1 `main.py` — carga de configuración e instanciación

La función `cargar_configuracion()` lee el JSON y termina el proceso con `sys.exit(1)` si el archivo no existe o tiene un error de sintaxis, lo que produce un mensaje claro en el log del servicio `systemd`:

```
def cargar_configuracion(ruta_config: str) -> dict:
    if not os.path.exists(ruta_config):
        print(f'[main] ERROR: {ruta_config} no encontrado')
        sys.exit(1)
    with open(ruta_config, 'r', encoding='utf-8') as archivo:
        try:
            return json.load(archivo)
        except json.JSONDecodeError as e:
            print(f'[main] ERROR en configuracion.json: {e}')
            sys.exit(1)
```

La función `construir_ruta()` genera rutas absolutas a partir de la ruta base del script y las claves del JSON, garantizando que las rutas funcionen independientemente del directorio desde el que se lanza el proceso:

```
RUTA_BASE = os.path.dirname(os.path.abspath(__file__))

def construir_ruta(config: dict, *claves) -> str:
    valor = config
    for clave in claves:
        valor = valor[clave]
    return os.path.join(RUTA_BASE, valor)
```

7.2 `MonitorUSB` — hilo de monitoreo con `pyudev`

El módulo `usb_monitor.py` contiene la clase `MonitorUSB`, encargada de detectar memorias USB conectadas a la Raspberry Pi. Esta detección se realiza en un hilo independiente para que la interfaz principal de la consola no se detenga mientras espera la conexión de un dispositivo.

```
self._hilo = threading.Thread(
    target=self._monitorear_udev,
    daemon=True
)
self._hilo.start()
```

El método `_monitorear_udev()` filtra por subsistema 'block' y tipo 'partition' para recibir únicamente eventos de particiones de almacenamiento masivo, ignorando otros dispositivos USB como teclados o gamepads:

```
def _monitorear_udev(self):
    import pyudev
    context = pyudev.Context()
    monitor = pyudev.Monitor.from_netlink(context)
    monitor.filter_by(subsystem='block', device_type='partition')
    for device in iter(monitor.poll, None):
        if device.action == 'add':
            time.sleep(1) # esperar montaje del kernel
            if self._montar(device.device_node):
                if self._emulador_ref and self._emulador_ref.proceso:
                    self._emulador_ref.proceso.terminate()
                    self._emulador_ref.proceso.wait()
                with self._lock:
                    self._usb_nuevo = True
```

7.3 MonitorUSB — copia de ROMs sin duplicados

Después de detectar y montar una memoria USB, el sistema debe buscar archivos compatibles y copiarlos a las carpetas locales del proyecto. Esto se realiza mediante el método `copiar_roms_de_usb()`. Para evitar copiar juegos repetidos, el sistema genera un conjunto con los nombres de archivos que ya existen en la carpeta destino. La comparación se realiza en minúsculas, por lo que archivos como Mario.nes y mario.nes se consideran equivalentes.

```
def copiar_roms_de_usb(self, ruta_roms_local: str) -> int:
    total = 0
    for consola, extensiones in self.EXTENSIONES_POR_CONSOLA.items():
        roms_en_usb = self._buscar_archivos(
            self.PUNTO_MONTAJE, extensiones
        )
        destino = os.path.join(ruta_roms_local, consola)
        os.makedirs(destino, exist_ok=True)
        ya_existen = {f.lower() for f in os.listdir(destino)}
        for ruta_rom in roms_en_usb:
            nombre = os.path.basename(ruta_rom)
            if nombre.lower() in ya_existen:
                continue # omitir duplicado
            shutil.copy2(ruta_rom, os.path.join(destino, nombre))
            ya_existen.add(nombre.lower())
            total += 1
    self._desmontar()
    return total
```

7.4 instalar.sh — detección de usuario y estructura de directorios

El script `instalar.sh` automatiza la preparación del sistema. Una parte importante del script es detectar correctamente qué usuario está ejecutando la instalación. Esto es necesario porque el script se ejecuta con `sudo`, pero los archivos del proyecto no deben quedar asignados solamente al usuario `root`.

```
if [ -n "${SUDO_USER:-}" ] && [ "${SUDO_USER:-}" != "root" ]; then
    USUARIO="${SUDO_USER}"
else
    USUARIO="$(logname 2>/dev/null || echo pi)"
fi
```

A continuación crea la estructura de directorios necesaria y asigna la propiedad al usuario real:

```
mkdir -p "$RUTA_SRC/roms/nes"
mkdir -p "$RUTA_SRC/roms/snes"
mkdir -p "$RUTA_SRC/roms/gba"
mkdir -p "$HOME_DIR/.mednafen"
mkdir -p /media/pi/usb_retro
```

Finalmente, el script asigna la propiedad de los archivos al usuario correspondiente. Esto permite que la aplicación pueda leer y escribir dentro de las carpetas del proyecto sin depender de permisos de administrador durante su ejecución normal.

```
chown -R "$USUARIO:$USUARIO" "$RUTA_PROYECTO"
```

8. Integración del sistema completo

La Figura 1 ilustra cómo main.py conecta todos los módulos del sistema. Las flechas indican referencias entre objetos: main crea cada instancia y pasa las referencias necesarias. El loop de Galeria es el único componente que corre indefinidamente; todos los demás módulos son llamados desde él o desde sus dependencias.

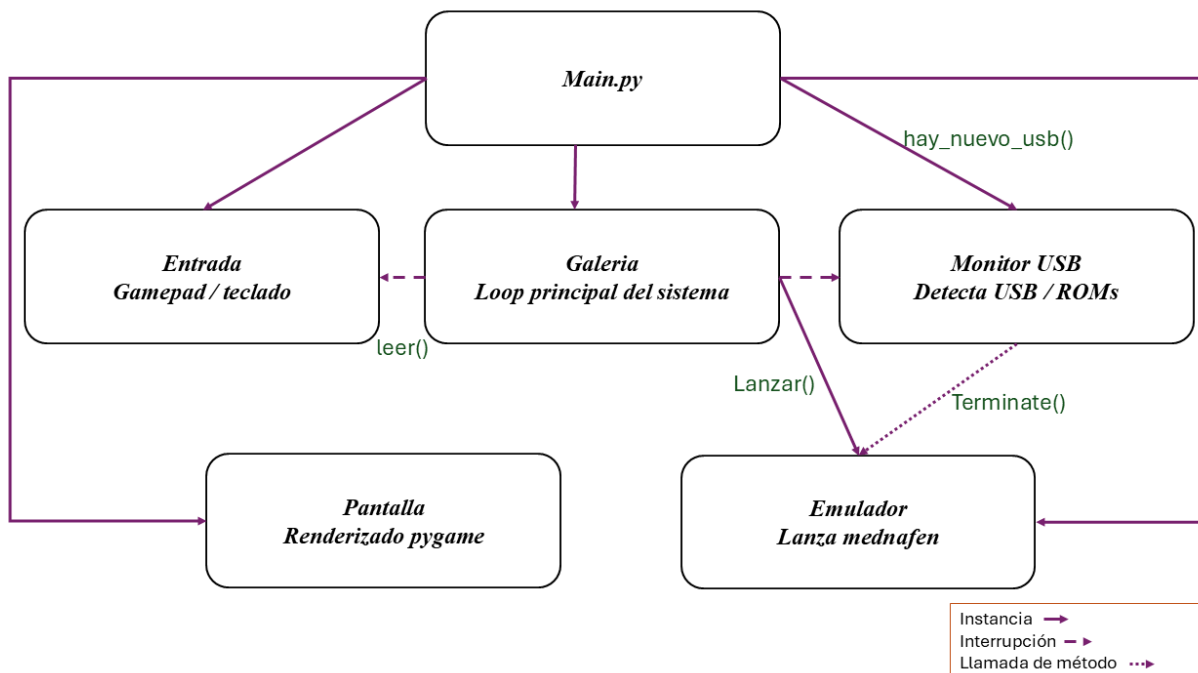


Figura 1. Diagrama de integración de módulos del sistema Consola Retro SDK

El flujo general de integración es el siguiente:

- El hilo de MonitorUSB detecta el evento 'add' de pyudev.

- Llama a `_montar()` para montar el USB en modo solo lectura.
- Si `Emulador.proceso` no es `None` (hay juego en curso), llama a `proceso.terminate()` y `.wait()`.
- Establece `_usb_nuevo = True` dentro del lock.
- En la siguiente iteración del loop de Galeria, `hay_nuevo_usb()` devuelve `True`.
- La galería muestra el mensaje de copia, llama a `usb_monitor.copiar_roms_de_usb()` y actualiza el catálogo.

9. Solución de problemas comunes

9.1 El servicio no arranca: verificar el log

Si la consola no inicia automáticamente después de encender la Raspberry Pi, lo primero que se debe revisar es el estado del servicio `consola-retro.service`. Para hacerlo se utiliza el siguiente comando:

```
sudo systemctl status consola-retro.service
cat ~/consola_retro.log
```

Si el error está relacionado con Python, rutas incorrectas, archivos faltantes o problemas con la configuración, normalmente aparecerá en este log. Por ejemplo, si `configuracion.json` no existe o tiene un error de sintaxis, `main.py` terminará la ejecución y el mensaje quedará registrado.

9.2 El USB no es detectado

Verificar que `pyudev` está instalado correctamente con `python3 -c 'import pyudev; print(pyudev.__version__)'`. Si `pyudev` no está disponible, el sistema cae al polling, que puede tardar hasta 2 segundos en detectar el USB. Verificar también que el usuario pertenece al grupo `plugdev` con `groups $USER`.

9.3 El USB se detecta pero no se monta

Verificar que el archivo `/etc/sudoers.d/consola_retro` existe y contiene las reglas de mount y umount sin contraseña. Verificar también que el directorio `/media/pi/usb_retro` existe con permisos correctos:

```
sudo cat /etc/sudoers.d/consola_retro
ls -la /media/pi/
```

9.4 El sistema no arranca automáticamente tras la instalación

Verificar que el servicio está habilitado con `systemctl is-enabled consola-retro.service`. Si responde `'disabled'`, habilitarlo manualmente:

```
sudo systemctl enable --now consola-retro.service
```

También verificar que no existe el archivo `/boot/NO_CONSOLA` ni `/boot/firmware/NO_CONSOLA`, ya que su presencia deshabilita el arranque automático.

9.5 Acceder al sistema para mantenimiento

Para acceder al sistema sin que la consola arranque, crear el archivo de control y reiniciar:

```
sudo touch /boot/NO_CONSOLA
```

```
sudo reboot
```

Al reiniciar, el servicio no arrancará y se podrá acceder a la terminal. Para volver al modo consola:

```
sudo rm /boot/NO_CONSOLA
sudo reboot
```

10. Resultados

Repositorio del proyecto: https://github.com/nasaruke/SDK_RetroGames

Video demostrativo: <https://youtu.be/pv0NvIFaCqE>

11. Conclusiones

En este proyecto se logró integrar una consola retro utilizando una Raspberry Pi como plataforma principal. A través del archivo main.py fue posible coordinar los módulos de pantalla, entrada, galería, emulador, arranque y monitoreo USB, manteniendo una estructura organizada donde cada componente cumple una función específica dentro del sistema. Como podemos describir para esta sección.

El uso de systemd permitió que la consola iniciara automáticamente al encender la Raspberry Pi, sin necesidad de ejecutar comandos manualmente desde la terminal. Esto fue importante para que el sistema se comportara como una consola dedicada y no como una computadora de propósito general.

La detección de memorias USB mediante pyudev permitió cargar nuevas ROMs de una forma más práctica para el usuario. En lugar de copiar archivos manualmente por SSH o desde la terminal, el sistema puede detectar una memoria conectada, montarla, buscar archivos compatibles y copiar únicamente las ROMs nuevas. Esto mejora la experiencia de uso y hace que la consola sea más sencilla de actualizar.

Finalmente, el script instalar.sh facilitó la configuración completa del sistema, ya que automatizó la instalación de dependencias, la creación de carpetas, la configuración de permisos, la preparación de Mednafen y el registro del servicio de arranque. Con esto, el proyecto puede reproducirse de forma más ordenada en una instalación limpia de Raspberry Pi OS Lite, reduciendo errores manuales y haciendo más sencillo preparar nuevas tarjetas microSD.

12. Cuestionario

Tabla 3. Cuestionario de evaluación

#	Pregunta
1	¿Por qué se utiliza doble buffer (display.flip) en lugar de actualizar la pantalla píxel a píxel?
2	¿Qué ventaja tiene encapsular todas las llamadas a pygame dentro de la clase Pantalla en lugar de llamar a pygame directamente desde cada módulo?
3	¿Por qué se define un umbral de 0.5 para los ejes analógicos en lugar de detectar cualquier valor distinto de cero? ¿Qué problema resuelve esto en la práctica?
4	Describe el problema del framebuffer compartido entre pygame y mednafen. ¿Por qué es necesario llamar a pygame.display.quit() antes de lanzar mednafen y cómo se recupera la pantalla al regresar?

5	¿Qué función cumple el <code>threading.Lock</code> en la clase <code>MonitorUSB</code> ? ¿Qué condición de carrera podría ocurrir si se eliminara el lock?
6	¿Por qué se utiliza <code>pyudev</code> para detectar la inserción de USB en lugar de revisar periódicamente el contenido de <code>/dev</code> ? Describe las ventajas del enfoque basado en eventos sobre el polling.

13. Bibliografía

[1] E. A. Lee y S. A. Seshia, Introduction to Embedded Systems: A Cyber-Physical Systems Approach, 2.a ed. Cambridge, MA: MIT Press, 2017.

[2] Lennart Poettering y Kay Sievers, "systemd System and Service Manager," freedesktop.org, 2024. [En línea]. Disponible en: <https://www.freedesktop.org/wiki/Software/systemd/>

[3] pyudev Contributors, "pyudev Documentation," 2024. [En línea]. Disponible en: <https://pyudev.readthedocs.io/>

[4] M. Kerrisk, The Linux Programming Interface. San Francisco, CA: No Starch Press, 2010.

[5] Python Software Foundation, "threading — Thread-based parallelism," Python 3.11 Documentation, 2024. [En línea]. Disponible en: <https://docs.python.org/3/library/threading.html>